



Приложение А. Математический минимум

Введение

Математика в этом курсе — не декорация. Она появляется тогда, когда без неё невозможно понять, **почему** система ведёт себя именно так. Почему косинусное сходство измеряет семантическую близость текстов? Почему $p\text{-value} = 0.03$ не означает, что гипотеза верна с вероятностью 97%? Почему `asyncio.Queue` с `maxsize=0` взрывается под нагрузкой, а с `maxsize=1000` — нет?

Это приложение — **три столпа**, на которых стоит вся математика курса: линейная алгебра (пространства данных), теория вероятностей (неопределённость и вывод), анализ сложности (предсказание поведения системы). Мы пойдём от определений к приложениям, с формальными выкладками и кодом на Python.

А.1. Линейная алгебра: векторы, матрицы, скалярное произведение

А.1.1. Вектор: от упорядоченного набора чисел до направления

Определение. Вектор $\mathbf{v}$ в n -мерном евклидовом пространстве $\mathbb{R}^n$ — это упорядоченный набор n вещественных чисел:

$$\mathbf{v} = (v_1, v_2, \dots, v_n)^T$$

В Python вектор — это `list[float]`, `tuple[float, ...]` или, что предпочтительнее, `numpy.ndarray`:

```
In [ ]: import numpy as np

v = np.array([1.0, 2.0, 3.0])      # вектор-столбец по умолчанию
w = np.array([0.5, -1.0, 2.0])
```

Размерность (dimensionality) — число n . В ML это число признаков

(features). Каждый объект (пользователь, товар, документ) — точка в $\mathbb{R}^n$.

Операции над векторами:

1. **Сложение** (покоординатное): $\mathbf{u} + \mathbf{v} = (u_1 + v_1, \dots, u_n + v_n)^T$ Геометрически: правило параллелограмма.
2. **Умножение на скаляр**: $\alpha \mathbf{v} = (\alpha v_1, \dots, \alpha v_n)^T$ Геометрически: растяжение ($|\alpha| > 1$) или сжатие ($|\alpha| < 1$), возможно с отражением ($\alpha < 0$).
3. **Норма (длина)**: $\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$ Это обобщение теоремы Пифагора на n измерений.

```
In [ ]: # Ручная реализация
def norm_l2(v: np.ndarray) -> float:
    return np.sqrt(np.sum(v ** 2))

# Или встроенная
np.linalg.norm(v) # 3.74165738677...
```

Единичный вектор (unit vector): $\|\mathbf{v}\| = 1$. Любой ненулевой вектор можно нормировать: $\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$

A.1.2. Матрица: линейное отображение

Определение. Матрица $A \in \mathbb{R}^{m \times n}$ — прямоугольная таблица из m строк и n столбцов:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

В Python:

```
In [ ]: A = np.array([
    [1, 2, 3],
    [4, 5, 6],
]) # shape: (2, 3)

# Доступ к элементам
A[1, 2] # 6 (вторая строка, третий столбец)
```

Умножение матрицы на вектор:

Если $A \in \mathbb{R}^{m \times n}$ и $\mathbf{x} \in \mathbb{R}^n$, то $A\mathbf{x} \in \mathbb{R}^m$:

$$(A\mathbf{x})_i = \sum_{j=1}^n a_{ij} x_j$$

Это **линейное отображение**: $A(\alpha \mathbf{x} + \beta \mathbf{y}) = \alpha A\mathbf{x} + \beta A\mathbf{y}$.

Умножение матриц: если $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, то $C = AB \in \mathbb{R}^{m \times p}$:

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

Сложность: $O(mnp)$. Для квадратных матриц $n \times n$: $O(n^3)$ (наивное умножение). Страссен даёт $O(n^{2.81})$, современные алгоритмы — $O(n^{2.37})$, но на практике BLAS использует оптимизированное $O(n^3)$.

```
In [ ]: A = np.array([[1, 2], [3, 4]])
        B = np.array([[5, 6], [7, 8]])

        # Матричное умножение
        C = A @ B # или np.dot(A, B)
        # [[19, 22],
        #   [43, 50]]
```

Транспонирование: A^T — матрица, где строки и столбцы меняются местами. $(A^T)_{ij} = a_{ji}$.

Свойство: $(AB)^T = B^T A^T$.

A.1.3. Скалярное произведение: проекция и угол

Определение. Скалярное (внутреннее) произведение векторов $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} = \sum_{i=1}^n u_i v_i$$

В Python:

```
In [ ]: np.dot(u, v)      # 1*0.5 + 2*(-1) + 3*2 = 0.5 - 2 + 6 = 4.5
        u @ v             # то же самое в NumPy
```

Геометрический смысл:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \cdot \|\mathbf{v}\| \cdot \cos \theta$$

где θ — угол между векторами. Это следует из **теоремы косинусов** в n -мерном пространстве.

Следствия:

1. Если $\mathbf{u} \perp \mathbf{v}$ (перпендикулярны), то $\langle \mathbf{u}, \mathbf{v} \rangle = 0$.
2. Если $\mathbf{u} = \mathbf{v}$, то $\langle \mathbf{u}, \mathbf{u} \rangle = \|\mathbf{u}\|^2$.
3. **Проекция** $\mathbf{v}$ на $\mathbf{u}$: $\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|^2} \mathbf{u}$.

Косинусное сходство (cosine similarity):

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} = \cos \theta$$

- $\text{sim} = 1$: векторы сонаправлены (одинаковое направление).
- $\text{sim} = 0$: ортогональны (независимы).
- $\text{sim} = -1$: противоположно направлены.

```
In [ ]: def cosine_similarity(u: np.ndarray, v: np.ndarray) -> float:
        return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

# Пример: "кошка" и "собака" в embedding-пространстве
cat_emb = np.array([0.2, 0.8, 0.1, -0.3]) # 4-мерный embedding
dog_emb = np.array([0.3, 0.7, 0.2, -0.2])
car_emb = np.array([-0.5, 0.1, 0.9, 0.4])

print(cosine_similarity(cat_emb, dog_emb)) # ~0.98 (близки)
print(cosine_similarity(cat_emb, car_emb)) # ~-0.15 (далеки)
```

Почему косинус, а не евклидово расстояние?

Евклидово расстояние: $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\| = \sqrt{\sum (u_i - v_i)^2}$.

Проблема: длина документа влияет на embedding. Длинный текст о «кошках» и короткий текст о «кошках» имеют разную норму вектора, но **одинаковое направление**. Косинусное сходство **инвариантно к масштабу** (нормирование делит на длину), поэтому измеряет

семантику, а не объём.

A.1.4. Embeddings: векторы как смысл

Word embedding — отображение слова в вектор $\mathbb{R}^d$, где d обычно 128, 256, 768, 1536. Близкие по смыслу слова — близкие векторы.

Sentence embedding — вектор целого предложения/документа. Получается усреднением word embeddings или через transformer (CLS-токен, mean pooling).

Применение в бэкенде:

```
In [ ]: from typing import Protocol
import numpy as np

class EmbeddingService(Protocol):
    async def embed(self, text: str) -> np.ndarray: ...

class SemanticSearch:
    def __init__(self, embedding_service: EmbeddingService):
        self._embeddings: list[tuple[str, np.ndarray]] = [] # (id, vector)
        self._service = embedding_service

    async def index(self, doc_id: str, text: str):
        vec = await self._service.embed(text)
        self._embeddings.append((doc_id, vec))

    async def search(self, query: str, top_k: int = 5) -> list[tuple[str,
        query_vec = await self._service.embed(query)

        # Линейный поиск: O(N * d)
        # Для миллионов документов используют FAISS, HNSW
        scores = [
            (doc_id, cosine_similarity(query_vec, vec))
            for doc_id, vec in self._embeddings
        ]
        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]
```

Матрица сходства: если у нас N документов, матрица попарных косинусов — это $N \times N$ матрица S , где $S_{ij} = \cos(\theta_{ij})$. Она симметрична ($S^T = S$) и положительно полуопределённая.

A.2. Теория вероятностей: матожидание, дисперсия, распределения

А.2.1. Случайная величина и пространство событий

Вероятностное пространство — тройка $(\Omega, \mathcal{F}, P)$:

- Ω — множество элементарных исходов.
- $\mathcal{F}$ — σ -алгебра событий (множество подмножеств Ω).
- $P: \mathcal{F} \rightarrow [0, 1]$ — вероятностная мера, $P(\Omega) = 1$.

Случайная величина (random variable) — функция $X: \Omega \rightarrow \mathbb{R}$, измеримая относительно $\mathcal{F}$. Проще: число, значение которого зависит от случайного исхода.

Пример: бросок кубика. $\Omega = \{1, 2, 3, 4, 5, 6\}$. $X(\omega) = \omega$ — случайная величина «число очков».

А.2.2. Математическое ожидание

Определение. Для дискретной случайной величины:

$$\mathbb{E}[X] = \sum_i x_i \cdot P(X = x_i)$$

Для непрерывной:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x) \, dx$$

где $f(x)$ — плотность распределения.

Интуиция: среднее значение при бесконечном числе повторений эксперимента. Закон больших чисел: $\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{n \rightarrow \infty} \mathbb{E}[X]$.

Свойства:

1. **Линейность:** $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$. Всегда, даже если X и Y зависимы.
2. **Для константы:** $\mathbb{E}[c] = c$.
3. **Для произведения:** $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ **только если** X и Y независимы.

```
In [ ]: # Эмпирическое ожидание (выборочное среднее)
samples = np.random.normal(loc=5.0, scale=2.0, size=100_000)
mean = np.mean(samples) # ~5.0
```

А.2.3. Дисперсия и стандартное отклонение

Дисперсия — мера разброса вокруг среднего:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Стандартное отклонение: $\sigma(X) = \sqrt{\text{Var}(X)}$. В тех же единицах, что и X .

Свойства:

1. $\text{Var}(aX + b) = a^2 \text{Var}(X)$. Сдвиг b не влияет.
2. Для независимых X, Y : $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.
3. Для зависимых: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$.

```
In [ ]: variance = np.var(samples, ddof=1) # выборочная дисперсия (несмещённая)
std = np.std(samples, ddof=1) # ~2.0
```

А.2.4. Основные распределения

Равномерное распределение $U(a, b)$

$$f(x) = \begin{cases} \frac{1}{b-a}, & x \in [a, b] \\ 0, & \text{иначе} \end{cases}$$
$$\mathbb{E}[X] = \frac{a+b}{2}, \quad \text{Var}(X) = \frac{(b-a)^2}{12}$$

Применение: генерация случайных чисел, jitter в retry-алгоритмах (Модуль 11).

```
In [ ]: np.random.uniform(0, 1, size=1000) # 1000 чисел из [0, 1)
```

Нормальное (гауссово) распределение $\mathcal{N}(\mu, \sigma^2)$

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$
$$\mathbb{E}[X] = \mu, \quad \text{Var}(X) = \sigma^2$$

Центральная предельная теорема (ЦПТ): если $X_1, \dots, X_n$ — независимые одинаково распределённые случайные величины с конечной дисперсией, то при $n \rightarrow \infty$:

$$\frac{\bar{X} - \mu}{\sigma / \sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

где $\bar{X} = \frac{1}{n} \sum X_i$.

Следствие: среднее достаточно большой выборки **всегда** нормально распределено, независимо от распределения самих величин. Это фундамент A/B тестирования.

```
In [ ]: # Демонстрация ЦПТ
samples = np.random.exponential(scale=2.0, size=(1000, 100)) # экспоненц
means = np.mean(samples, axis=1) # средние по 1000 выборкам размера 100
# means ~ N(2, 2^2/100) = N(2, 0.04)
```

Биномиальное распределение $\text{Bin}(n, p)$

Число успехов в n независимых испытаниях Бернулли с вероятностью успеха p .

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k} \quad \mathbb{E}[X] = np, \quad \text{Var}(X) = np(1-p)$$

Применение: CTR (click-through rate) — n показов, k кликов, $\hat{p} = k/n$.

Пуассоновское распределение $\text{Pois}(\lambda)$

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad \mathbb{E}[X] = \lambda, \quad \text{Var}(X) = \lambda$$

Применение: интенсивность поступления запросов (RPS) в теории очередей (Модуль 2, 9). Если запросы приходят независимо с постоянной интенсивностью λ , число запросов за интервал t — $\text{Pois}(\lambda t)$.

A.2.5. A/B тестирование: от теории к коду

Постановка: две версии (A — control, B — treatment). Хотим понять, отличаются ли они по метрике M (например, conversion rate).

Нулевая гипотеза H_0 : $\mu_A = \mu_B$ (различия нет, наблюдаемые отклонения — случайны). **Альтернативная H_1 :** $\mu_A \neq \mu_B$.

Z-тест для пропорций (бинарная метрика):

Пусть в группе A: n_A пользователей, x_A конверсий. В группе B: n_B , x_B .

$$\hat{p}_A = \frac{x_A}{n_A}, \quad \hat{p}_B = \frac{x_B}{n_B}$$

$$\hat{p} = \frac{x_A + x_B}{n_A + n_B}$$

Статистика:

$$Z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_A} + \frac{1}{n_B}\right)}}$$

При H_0 верно, $Z \sim \text{mathcal{N}}(0, 1)$ (по ЦПТ).

p-value: $P(|Z_{\text{theory}}| > |Z_{\text{observed}}| \mid H_0)$ — вероятность получить такое или более экстремальное отклонение, **если H_0 верна**.

Ошибки:

	H_0 верна	H_0 ложна
Отклонить H_0	Ошибка I рода (α)	Верно
Не отклонить H_0	Верно	Ошибка II рода (β)

- α — уровень значимости (обычно 0.05).
- $1 - \beta$ — мощность (обычно 0.8).

Критическое заблуждение: p-value — это **не** вероятность того, что H_0 верна. Это вероятность данных при условии H_0 .

```
In [ ]: from scipy import stats
import numpy as np

def ab_test_proportions(
    conversions_a: int, total_a: int,
    conversions_b: int, total_b: int,
    alpha: float = 0.05,
) -> dict:
    p_a = conversions_a / total_a
    p_b = conversions_b / total_b
    p_pooled = (conversions_a + conversions_b) / (total_a + total_b)

    se = np.sqrt(p_pooled * (1 - p_pooled) * (1/total_a + 1/total_b))
    z = (p_a - p_b) / se

    # Двусторонний тест
    p_value = 2 * (1 - stats.norm.cdf(abs(z)))
```

```

# Доверительный интервал для разности
diff = p_a - p_b
se_diff = np.sqrt(p_a*(1-p_a)/total_a + p_b*(1-p_b)/total_b)
z_crit = stats.norm.ppf(1 - alpha/2)
ci_lower = diff - z_crit * se_diff
ci_upper = diff + z_crit * se_diff

return {
    "p_a": p_a,
    "p_b": p_b,
    "difference": diff,
    "z_score": z,
    "p_value": p_value,
    "ci_95": (ci_lower, ci_upper),
    "significant": p_value < alpha,
}

# Пример
result = ab_test_proportions(
    conversions_a=50, total_a=1000, # 5.0%
    conversions_b=65, total_b=1000, # 6.5%
)
# p_value ~ 0.08, не значимо на уровне 0.05
# Нужно больше данных!

```

Доверительный интервал: интервал $[L, U]$, который с вероятностью $1-\alpha$ покрывает истинное значение параметра. Если 95% ДИ для разности не включает 0 — различие значимо.

А.3. Сложность алгоритмов: О-нотация и амортизированный анализ

А.3.1. О-нотация: асимптотический рост

Определение. $f(n) = O(g(n))$, если $\exists C > 0, n_0 > 0$ такие, что:

$$\forall n \geq n_0: |f(n)| \leq C \cdot |g(n)|$$

Интуиция: при достаточно больших n функция f растёт не быстрее g (с точностью до константы).

Примеры:

$f(n)$	$O(\cdot)$	Почему
$3n^2 + 2n + 5$	$O(n^2)$	Квадратичный член доминирует
$100n + \log n$	$O(n)$	Линейный доминирует

$f(n)$	$O(\cdot)$	Почему
$2^n + n^{100}$	$O(2^n)$	Экспонента доминирует над полиномом
5	$O(1)$	Константа

Иерархия скоростей роста:

$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$

Почему это важно для бэкенда:

Операция	Сложность	Пример
Доступ по индексу в <code>list</code>	$O(1)$	<code>arr[5]</code>
Поиск в <code>list</code> (линейный)	$O(n)$	<code>x in arr</code>
Поиск в <code>set</code> / <code>dict</code>	$O(1)$ среднее, $O(n)$ худшее	<code>x in hash_set</code>
Сортировка	$O(n \log n)$	<code>sorted(arr)</code> (Timsort)
Поиск в БД без индекса	$O(n)$	<code>SELECT * FROM users WHERE email = 'a'</code>
Поиск в БД с B-Tree индексом	$O(\log n)$	Тот же запрос с <code>INDEX</code>

A.3.2. Анализ Python-структур данных

`list` (динамический массив):

- `append` — **амортизированно** $O(1)$. Иногда $O(n)$ при расширении, но в среднем $O(1)$.
- `insert(0, x)` — $O(n)$ (сдвиг всех элементов).
- `pop()` — $O(1)$. `pop(0)` — $O(n)$.
- `x in lst` — $O(n)$.

`dict` (хеш-таблица):

- `get`, `set`, `delete` — $O(1)$ среднее, $O(n)$ худшее (коллизии).
- `keys()`, `values()` — $O(1)$ (возвращают view, не копируют).

`set` (хеш-таблица):

- `add`, `remove`, `in` — $O(1)$ среднее.

- `union`, `intersection` — $O(|A| + |B|)$.

`collections.deque` (двусвязный список):

- `append`, `appendleft`, `pop`, `popleft` — $O(1)$.
- `x in deque` — $O(n)$.

А.3.3. Амортизированный анализ: банковский метод

Проблема: `list.append` иногда требует перевыделения памяти и копирования всех элементов ($O(n)$). Но «в среднем» он быстрый. Как формализовать?

Амортизированный анализ оценивает **последовательность** операций, а не одну.

Банковский метод: каждая `append` «кладёт в банк» 1 условную монету. Когда массив переполняется (ёмкость C), нужно C операций для копирования. Но в банке накоплено C монет (по одной за каждое `append` с момента последнего расширения). Итого: n операций `append` стоят $O(n)$, амортизированно $O(1)$ на операцию.

Формально: пусть Φ — потенциал структуры. Амортизированная стоимость:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

где c_i — реальная стоимость i -й операции, $\Phi(D_i)$ — потенциал состояния после операции.

Для `list`: $\Phi = 2n - C$, где n — число элементов, C — ёмкость. Обычно $C \approx 2n$ (Python увеличивает ёмкость с запасом).

А.3.4. Применение к асинхронным системам

Event Loop (Модуль 2):

Event loop — это структура данных (heap + deque + selector). Одна итерация:

1. Переместить просроченные задачи из `scheduled queue` в `ready queue` — $O(k \log m)$, где k — число просроченных, m — размер

scheduled.

2. Выполнить задачи из ready queue — $O(1)$ на задачу.
3. `selector.select()` — $O(\text{число готовых дескрипторов})$ (epoll).

Asyncio Queue (Модуль 2):

`Queue(maxsize=N)` — реализована на `collections.deque`. `put` и `get` — $O(1)$. Но если `maxsize` достигнут, `put` блокирует корутину. Очередь ожидания корутин — `deque`, операции $O(1)$.

База данных: B-Tree индекс (Модуль 6):

B-Tree — сбалансированное дерево поиска. Высота $h = O(\log_B n)$, где B — порядок (число ключей в узле, обычно соответствует размеру страницы диска, 4–16 КБ). Поиск, вставка, удаление — $O(\log n)$ дисковых операций.

Почему B-Tree, а не бинарное дерево? Бинарное дерево: $O(\log_2 n)$ операций, каждая — случайный доступ к памяти (кэш-промах). B-Tree: $O(\log_B n)$ операций, но каждая — последовательное чтение страницы (кэш-хит). При $n = 10^9$, $B = 1000$: высота B-Tree ≈ 3 , бинарного ≈ 30 .

А.3.5. Практика: профилирование и оценка

```
In [ ]: import time

def measure(func, *args, n=5):
    """Измеряет время выполнения, убирая выбросы."""
    times = []
    for _ in range(n + 2):
        start = time.perf_counter()
        func(*args)
        times.append(time.perf_counter() - start)
    times.sort()
    return sum(times[1:-1]) / n # убираем min и max

# Сравнение list vs set lookup
data = list(range(100_000))
data_set = set(data)

# O(n) vs O(1)
print(measure(lambda: 99_999 in data, n=10)) # ~1.5 мс
print(measure(lambda: 99_999 in data_set, n=10)) # ~0.05 мкс
# Разница в 30 000 раз!
```

Big-O — это не всё. Константы и hidden factors важны:

- $O(n)$ с константой 1 быстрее $O(\log n)$ с константой 100 при $n < 1000$.
- Кэш-локальность может перевесить асимптотику (B-Tree vs хеш-таблица при малом n).
- В Python операции на уровне C (NumPy, `pydantic-core`) имеют константу на порядок меньше, чем чистый Python.

Итог приложения А

Концепция	Формула / Определение	Применение в курсе
Вектор	$\mathbf{v} \in \mathbb{R}^n$	Embeddings, feature vectors
Норма	$\ \mathbf{v}\ = \sqrt{\sum v_i^2}$	Нормализация векторов
Скалярное произведение	$\langle \mathbf{u}, \mathbf{v} \rangle = \sum u_i v_i$	Cosine similarity семантического поиска
Косинусное сходство	$\cos\theta = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\ \mathbf{u}\ \ \mathbf{v}\ }$	Поиск похожих документов, рекомендации
Матожидание	$\mathbb{E}[X] = \sum x_i P(X=x_i)$	Средняя latency, RPS
Дисперсия	$\text{Var}(X) = \mathbb{E}[(X - \mu)^2]$	Вариативность latency, p95/p99
Нормальное распределение	$\mathcal{N}(\mu, \sigma^2)$	ЦПТ, A/B тесты, Z-статистика
Биномиальное	$\text{Bin}(n, p)$	CTR, conversion rate
Пуассоновское	$\text{Pois}(\lambda)$	Интенсивность запросов, теория очередей
Z-тест	$Z = \frac{\hat{p}_A - \hat{p}_B}{SE}$	A/B тестирование моделей
p-value	$P(\text{данные} \mid H_0)$	Статистическая значимость
O-нотация	$f(n) = O(g(n))$	Оценка производительности структур данных
Амортизированный анализ	$\hat{c}_i = c_i + \Delta\Phi$	<code>list.append</code> , <code>deque</code> operations
B-Tree	$O(\log_B n)$	Индексы PostgreSQL, поиск в БД

Этот минимум — фундамент, на котором строятся все математические рассуждения в курсе. От векторного представления слов до статистической значимости A/B теста и от оценки latency event loop до анализа индексов базы данных.